



Application Penetration Testing Report

Base URL: 10kfpomis.dac.gov.in

Ticket Creation Date: 10-07-26 06:13 pm

Application Security and Audit Division
(Application Security Group)
National Informatics Centre
A – Block, CGO Complex, Lodhi Road, New Delhi - 110003.

Contributions

S. No.	Name	Role
1.	Rakhi Tiwari	Auditor
2.	Sudhir Kumar Verma	Reviewer

Key Findings

Vulnerability List

Assign Number: A20260708012		
Base URL: 10kfpomis.dac.gov.in		
S. No.	Incident URL	Vulnerability Name
1.	https://10kfpomis.dac.gov.in/MainetService/assets/libs/fo	Vulnerable and Outdated Components
2.	https://10kfpomis.dac.gov.in/MainetService/Home.html	Host Header Injection
3.	https://10kfpomis.dac.gov.in	Insecure Cookie Attributes
4.	https://10kfpomis.dac.gov.in	Server Fingerprinting & Framework Disclosure
5.	https://10kfpomis.dac.gov.in	Weak Ciphers Enabled
6.	https://10kfpomis.dac.gov.in/MainetService/Home.html	Missing CAPTCHA
7.	https://10kfpomis.dac.gov.in	Missing Security Response Header

1. Vulnerability Name: Vulnerable and Outdated Components

Incident URL:

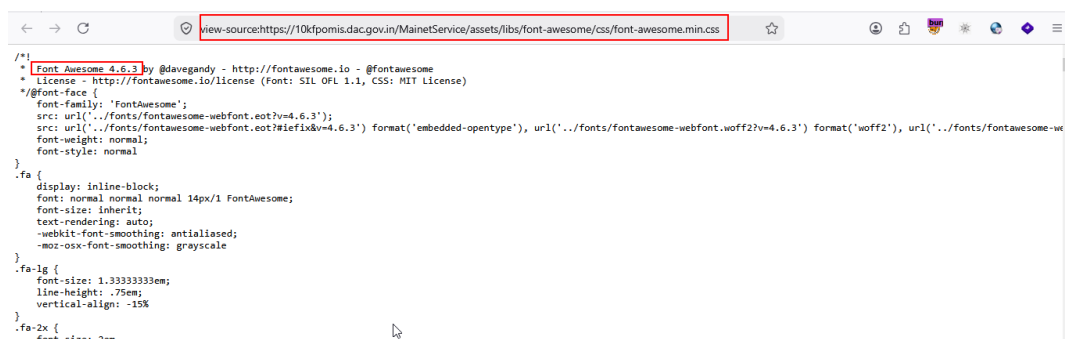
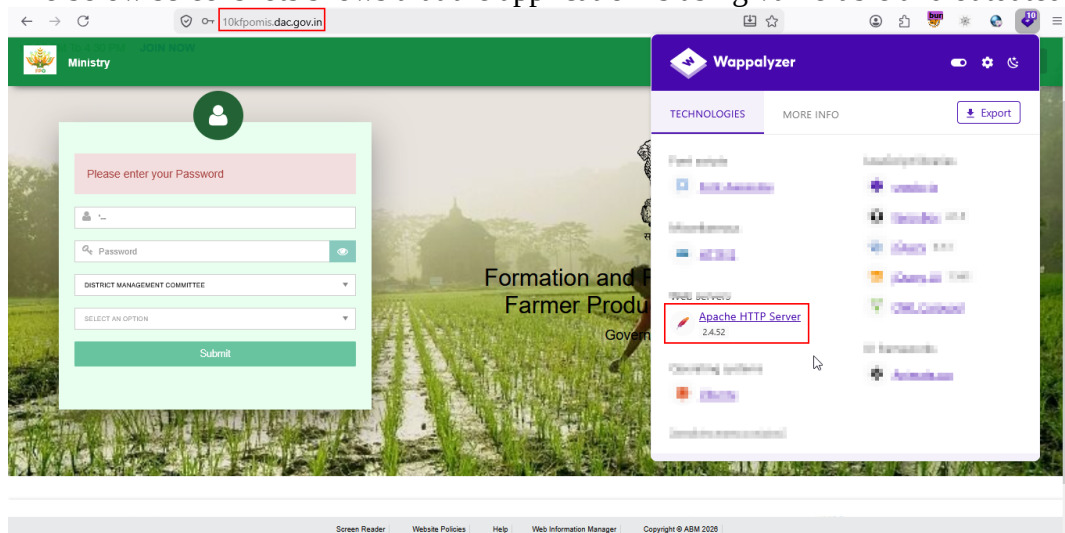
<https://10kfpomis.dac.gov.in/MainetService/assets/libs/font-awesome/css/font-awesome.min.css>

Severity:

Medium

Proof of Concept:

The below screenshots shows that the application is using vulnerable and outdated component.



Description:

The application is using outdated and deprecated components that are no longer actively maintained or supported by the vendor. These components may lack recent security updates and improvements.

Impact:

Deprecated components increase the risk of security vulnerabilities, compatibility issues, and instability. Attackers may exploit known weaknesses in these components to compromise the application or gain unauthorized access.

Recommendations:

Ensure that all server software, frameworks, and dependencies are regularly updated to their latest stable versions by applying security patches in a timely manner. Implement a structured patch management process to continuously monitor vendor advisories, track versions, and replace any deprecated or end-of-life components with actively supported alternatives to maintain a secure and stable environment.

Other Links:

<https://10kfpomis.dac.gov.in/MainetService/assets/libs/lightbox/jquery.fancybox.css>

<https://10kfpomis.dac.gov.in/MainetService/Home.html>

2. Vulnerability Name: Host Header Injection

Incident URL:

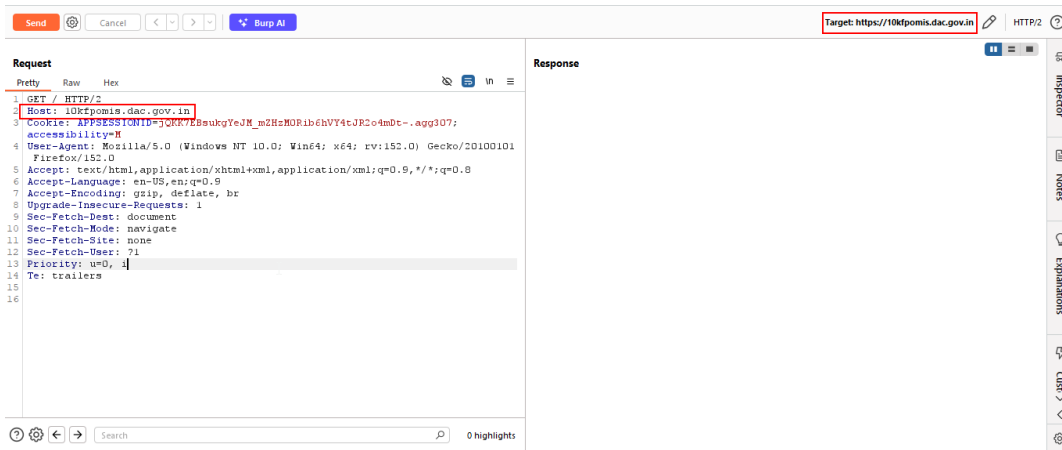
<https://10kfpomis.dac.gov.in/MainetService/Home.html>

Severity:

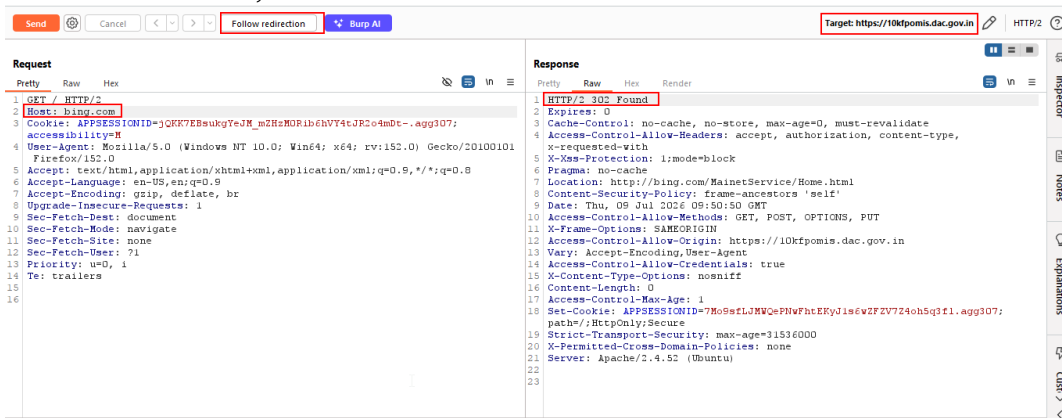
Medium

Proof of Concept:

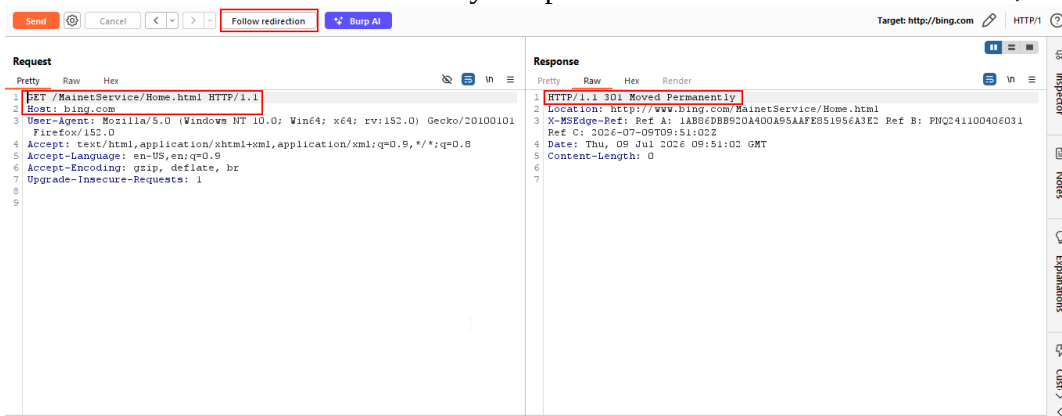
Capture the traffic using Burp Proxy and send the request to Repeater, as shown in the screenshot below.



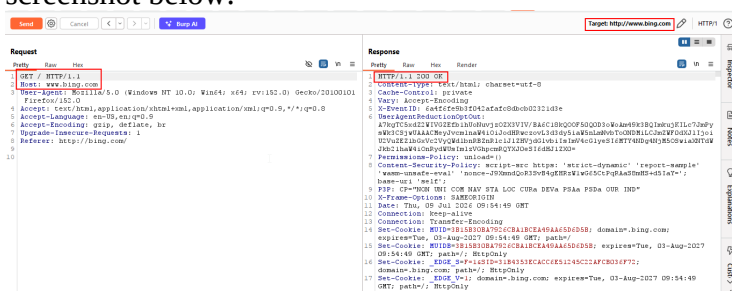
Modify the hostname and send the request. In the response, a 302 Found status is returned and click on "Follow redirection," as shown in the screenshot below.



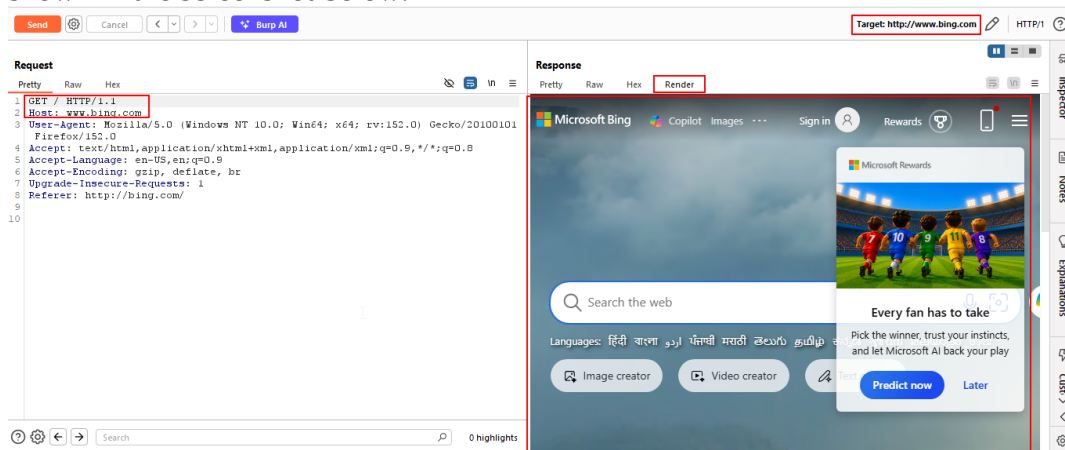
Received a “301 Moved Permanently” response. Click on “Follow Redirection,” as shown below.



In the response, we received a “200 OK” status and application redirected to bing.com, as shown in the screenshot below.



Then click on “Render” for further confirmation and application successfully redirected to bing.com , as shown in the screenshot below.



Description:

Host Header Injection is a web application vulnerability that occurs when an application improperly trusts and processes the HTTP Host header supplied by the client.

Impact:

Exploitation of this vulnerability can result in serious security issues such as password reset poisoning, web cache poisoning, open redirects, phishing attacks, bypass of security controls, and in some cases server-side request forgery (SSRF). These attacks may lead to credential theft, session hijacking, or unauthorized access.

Recommendations:

To mitigate Host Header Injection, applications should strictly validate and whitelist allowed hostnames, avoid using the Host header for security-sensitive operations, and generate absolute URLs from trusted server-side configurations. Additional protections include proper web server configuration, deploying WAF rules, and performing regular security assessments.

Other Links:

3. Vulnerability Name: Insecure Cookie Attributes

Incident URL:

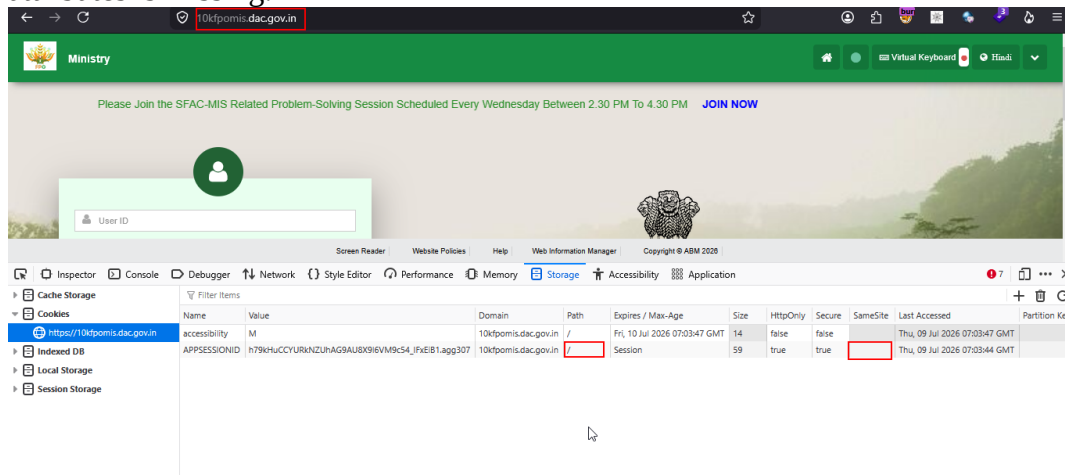
<https://10kfpomis.dac.gov.in>

Severity:

Low

Proof of Concept:

The screenshot below shows that the cookie's Path attribute is set to the root path (/), and the SameSite attributes is missing.



Description:

The screenshot below shows that the cookie's Path attribute is set to the root path (/), making the cookie accessible across the entire application. Additionally, the SameSite attribute is missing, meaning the browser does not have explicit instructions to restrict when the cookie is sent with cross-site requests.

Impact:

A cookie with a root (/) path is sent to all application paths, increasing its exposure if multiple components share the same domain. The absence of the SameSite attribute can allow the cookie to be included in cross-site requests, increasing the risk of Cross-Site Request Forgery (CSRF) attacks and unintended information disclosure.

Recommendations:

Set the cookie's Path attribute to the most restrictive path required by the application instead of using the root path wherever possible. Configure the SameSite attribute to Lax or Strict based on the application's functionality. If cross-site cookie usage is required, use SameSite=None together with the Secure attribute to ensure the cookie is transmitted only over HTTPS.

Other Links:

4. Vulnerability Name: Server Fingerprinting & Framework Disclosure

Incident URL:

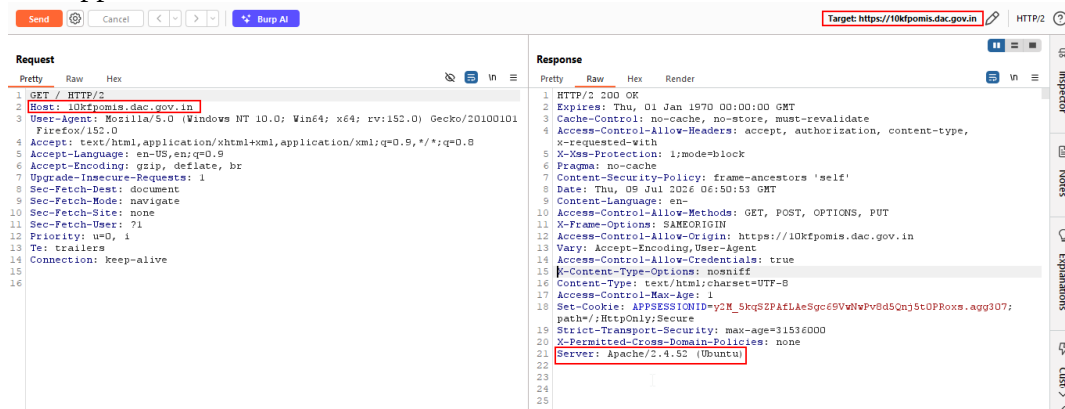
<https://10kfpomis.dac.gov.in>

Severity:

Low

Proof of Concept:

The application discloses the name and version of the server as shown in the below screenshots.



Description:

The application discloses the server and framework details like name and version to the end user.

Impact:

The application discloses the server and framework details like name and version to the end user. A malicious user can use this information to launch further targeted attacks.

Recommendations:

The application should minimize information disclosure by hiding or modifying server headers that reveal technology or version details, disabling verbose error messages and stack traces, and ensuring that HTTP responses and error pages do not expose sensitive system information. These security measures should be consistently implemented throughout the application.

Other Links:

5. Vulnerability Name: Weak Ciphers Enabled

Incident URL:

<https://10kfpomis.dac.gov.in>

Severity:

Low

Proof of Concept:

The cipher suites highlighted in the screenshot below are identified as weak.

```
Start 2026-07-09 04:10:22 → 164.100.252.244:443 (10kfpomis.dac.gov.in) ←
rDNS (164.100.252.244): --
Service detected: HTTP

Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      not offered (OK)
TLS 1      not offered
TLS 1.1    not offered
TLS 1.2    offered (OK)
TLS 1.3    offered (OK): final
QUIC       not offered or timed out
NPN/SPDY   not offered
ALPN/HTTP2 h2, http/1.1 (offered)

Testing cipher categories

NULL ciphers (no encryption)      not offered (OK)
Anonymous NULL Ciphers (no authentication) not offered (OK)
Export ciphers (w/o ADH+NULL)     not offered (OK)
LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export) not offered (OK)
Triple DES Ciphers / IDEA         not offered
Obsoleted CBC ciphers (AES, ARIA etc.) offered
Strong encryption (AEAD ciphers) with no FS offered (OK)
Forward Secrecy strong encryption (AEAD ciphers) offered (OK)
```

TLSv1.2 (no server order, thus listed by strength)					
xc030	ECDSHA-RSA-AES256-GCM-SHA384	ECDSHA 521	AESGCM	256	TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
xc028	ECDSHA-RSA-AES256-SHA384	ECDSHA 521	AES	256	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
xc014	ECDSHA-RSA-AES256-SHA	ECDSHA 521	AES	256	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
x9f	DHE-RSA-AES256-GCM-SHA384	DH 2048	AESGCM	256	TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
xc0a8	ECDSHA-RSA-CHACHA20-POLY1305	ECDSHA 521	ChaCha20	256	TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
xc0aa	DHE-RSA-CHACHA20-POLY1305	DH 2048	ChaCha20	256	TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256
xc0a3	DHE-RSA-AES256-CCM8	DH 2048	AESCCM8	256	TLS_DHE_RSA_WITH_AES_256_CCM_8
xc09f	DHE-RSA-AES256-CCM	DH 2048	AESCCM	256	TLS_DHE_RSA_WITH_AES_256_CCM
x6b	DHE-RSA-AES256-SHA256	DH 2048	AES	256	TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
x39	DHE-RSA-AES256-SHA	DH 2048	AES	256	TLS_DHE_RSA_WITH_AES_256_CBC_SHA
xc077	ECDSHA-RSA-CAMELLIA256-SHA384	ECDSHA 521	Camellia	256	TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384
xc4	DHE-RSA-CAMELLIA256-SHA256	DH 2048	Camellia	256	TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA256
x88	DHE-RSA-CAMELLIA256-SHA	DH 2048	Camellia	256	TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA
x9d	AES256-GCM-SHA384	RSA	AESGCM	256	TLS_RSA_WITH_AES_256_GCM_SHA384
xc0a1	AES256-CCM8	RSA	AESCCM8	256	TLS_RSA_WITH_AES_256_CCM_8
xc09d	AES256-CCM	RSA	AESCCM	256	TLS_RSA_WITH_AES_256_CCM
x3d	AES256-SHA256	RSA	AES	256	TLS_RSA_WITH_AES_256_CBC_SHA256
x35	AES256-SHA	RSA	AES	256	TLS_RSA_WITH_AES_256_CBC_SHA
xc0	CAMELLIA256-SHA256	RSA	Camellia	256	TLS_RSA_WITH_CAMELLIA_256_CBC_SHA256
x84	CAMELLIA256-SHA	RSA	Camellia	256	TLS_RSA_WITH_CAMELLIA_256_CBC_SHA
xc051	ARIA256-GCM-SHA384	RSA	ARIAGCM	256	TLS_RSA_WITH_ARIA_256_GCM_SHA384
xc053	DHE-RSA-ARIA256-GCM-SHA384	DH 2048	ARIAGCM	256	TLS_DHE_RSA_WITH_ARIA_256_GCM_SHA384
xc061	ECDSHA-RSA-AES256-GCM-SHA384	ECDSHA 521	ARIAGCM	256	TLS_ECDHE_RSA_WITH_ARIA_256_GCM_SHA384
xc02f	ECDSHA-RSA-AES128-GCM-SHA256	ECDSHA 521	AESGCM	128	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
xc027	ECDSHA-RSA-AES128-SHA256	ECDSHA 521	AES	128	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256

xc02f	ECDSHA-RSA-AES128-GCM-SHA256	ECDSHA 521	AESGCM	128	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
xc027	ECDSHA-RSA-AES128-SHA256	ECDSHA 521	AES	128	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
xc013	ECDSHA-RSA-AES128-SHA	ECDSHA 521	AES	128	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
x9e	DHE-RSA-AES128-GCM-SHA256	DH 2048	AESGCM	128	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
xc0a2	DHE-RSA-AES128-CCM8	DH 2048	AESCCM8	128	TLS_DHE_RSA_WITH_AES_128_CCM_8
xc09e	DHE-RSA-AES128-CCM	DH 2048	AESCCM	128	TLS_DHE_RSA_WITH_AES_128_CCM
xc0a0	AES128-CCM8	RSA	AESCCM8	128	TLS_RSA_WITH_AES_128_CCM_8
xc09c	AES128-CCM	RSA	AESCCM	128	TLS_RSA_WITH_AES_128_CCM
x67	DHE-RSA-AES128-SHA256	DH 2048	AES	128	TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
x33	DHE-RSA-AES128-SHA	DH 2048	AES	128	TLS_DHE_RSA_WITH_AES_128_CBC_SHA
xc076	ECDSHA-RSA-CAMELLIA128-SHA256	ECDSHA 521	Camellia	128	TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256
xbe	DHE-RSA-CAMELLIA128-SHA256	DH 2048	Camellia	128	TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA256
x45	DHE-RSA-CAMELLIA128-SHA	DH 2048	Camellia	128	TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA
x9c	AES128-GCM-SHA256	RSA	AESGCM	128	TLS_RSA_WITH_AES_128_GCM_SHA256
x3c	AES128-SHA256	RSA	AES	128	TLS_RSA_WITH_AES_128_CBC_SHA256
x2f	AES128-SHA	RSA	AES	128	TLS_RSA_WITH_AES_128_CBC_SHA
xba	CAMELLIA128-SHA256	RSA	Camellia	128	TLS_RSA_WITH_CAMELLIA_128_CBC_SHA256
x41	CAMELLIA128-SHA	RSA	Camellia	128	TLS_RSA_WITH_CAMELLIA_128_CBC_SHA
xc050	ARIA128-GCM-SHA256	RSA	ARIAGCM	128	TLS_RSA_WITH_ARIA_128_GCM_SHA256
xc052	DHE-RSA-ARIA128-GCM-SHA256	DH 2048	ARIAGCM	128	TLS_DHE_RSA_WITH_ARIA_128_GCM_SHA256
xc060	ECDSHA-RSA-ARIA128-GCM-SHA256	ECDSHA 521	ARIAGCM	128	TLS_ECDHE_RSA_WITH_ARIA_128_GCM_SHA256
TLSv1.3 (no server order, thus listed by strength)					
x1302	TLS_AES_256_GCM_SHA384	ECDSHA 253	AESGCM	256	TLS_AES_256_GCM_SHA384
x1303	TLS_CHACHA20_POLY1305_SHA256	ECDSHA 253	ChaCha20	256	TLS_CHACHA20_POLY1305_SHA256
x1301	TLS_AES_128_GCM_SHA256	ECDSHA 253	AESGCM	128	TLS_AES_128_GCM_SHA256

Description:

Weak cipher suites are outdated or insecure encryption algorithms used in SSL/TLS communication. These ciphers use weak cryptographic methods that can be easily broken by attackers, reducing the overall security of data in transit between client and server.

Impact:

If weak cipher suites are enabled, attackers may be able to decrypt sensitive data, perform man-in-the-middle attacks, or downgrade secure connections. This can lead to exposure of login credentials, personal data, and other confidential information.

Recommendations:

Disable obsolete CBC-based cipher suites, remove support for any weak or vulnerable TLS configurations, and enable only strong, modern ciphers such as TLS 1.2+ with AES-GCM or TLS 1.3 default suites. Additionally, regularly review TLS settings to maintain compliance with current security standards.

Other Links:

6. Vulnerability Name: Missing CAPTCHA

Incident URL:

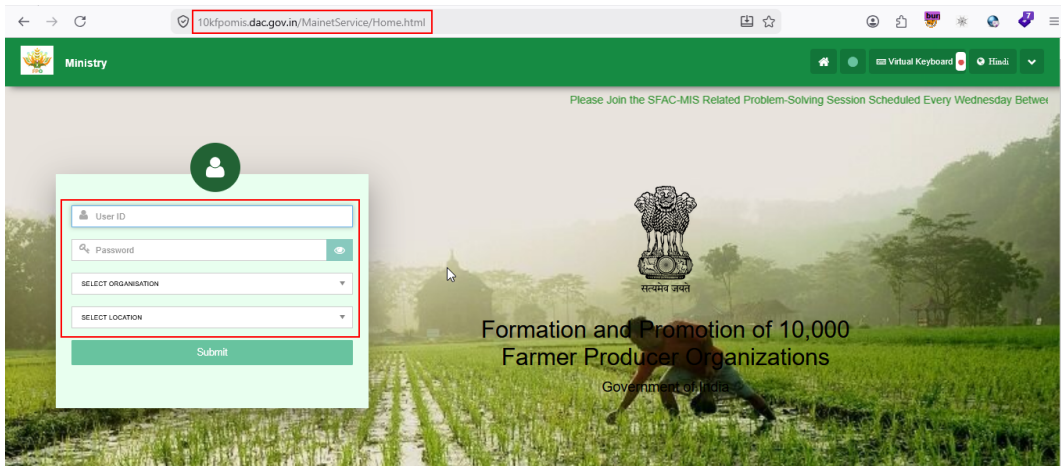
<https://10kfpomis.dac.gov.in/MainetService/Home.html>

Severity:

Low

Proof of Concept:

Navigate to the highlighted URL and observe that no CAPTCHA is present on the page, as shown in the screenshot below.



Description:

A Missing CAPTCHA vulnerability occurs when a web application does not implement a CAPTCHA mechanism on forms or sensitive actions (such as login, registration, password reset, or contact forms). Without CAPTCHA, automated bots can interact with the application without any challenge to verify that the user is human.

Impact:

Attackers can exploit the absence of CAPTCHA to perform automated attacks such as brute-force login attempts, credential stuffing, spam submissions, fake account creation, and denial-of-service activities. This can lead to unauthorized access, resource exhaustion, and degradation of service quality.

Recommendations:

Implement a reliable CAPTCHA solution (such as reCAPTCHA, hCaptcha, or similar) on forms and actions vulnerable to automation. Additionally, use rate limiting, account lockout policies, bot detection mechanisms, and monitoring to provide layered protection against automated abuse.

Other Links:

7. Vulnerability Name: Missing Security Response Header

Incident URL:

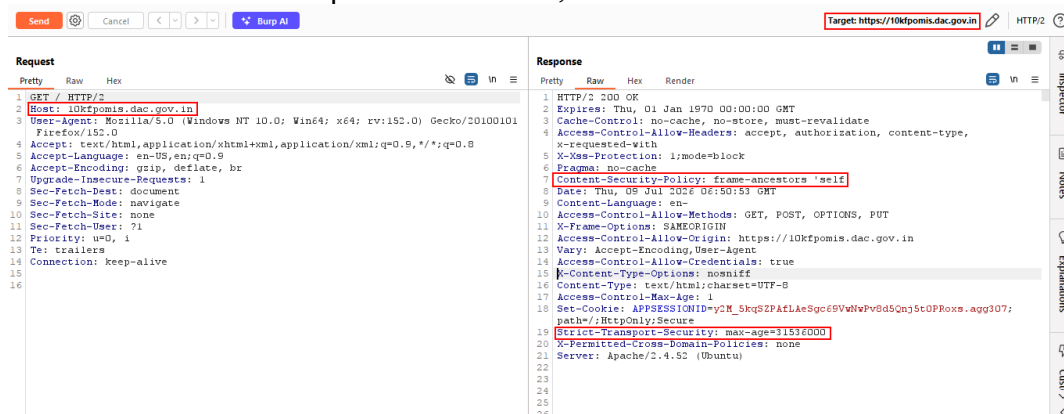
<https://10kfpomis.dac.gov.in>

Severity:

Low

Proof of Concept:

The screenshot below shows that the Content-Security-Policy header is missing the object-src and script-src directives. Additionally, the Strict-Transport-Security (HSTS) header is missing the includeSubDomains and preload directives, as shown in the screenshot below.



Description:

The application's HTTP security headers are not fully configured. The Content-Security-Policy (CSP) header is missing the object-src and script-src directives, and the Strict-Transport-Security (HSTS) header does not include the includeSubDomains and preload directives. These missing directives reduce the effectiveness of browser-enforced security protections.

Impact:

An incomplete CSP may allow the browser to load scripts or plugin-based content from unintended sources, increasing the risk of client-side attacks such as cross-site scripting (XSS). Similarly, an incomplete HSTS policy may leave subdomains vulnerable to downgrade or man-in-the-middle attacks, as they are not forced to use HTTPS.

Recommendations:

Configure the Content-Security-Policy header to include appropriate script-src and object-src directives, allowing only trusted sources and preferably setting object-src 'none' if plugin content is unnecessary. Update the Strict-Transport-Security header to include the includeSubDomains and preload directives after confirming that all subdomains support HTTPS and meet the requirements for HSTS preloading.

Other Links: